Triage de alertas SOC sobre 2M de eventos (Precinct6): propuesta de proyecto

Author Name
Curso: Aprendizaje Automático — Semestre 8

***Resumen*—Los SOC se ahogan en alertas: 2,011,674 eventos en 6 días con solo 0.38 % malicioso (IR 245:1). Proponemos construir un sistema de triage que clasifique cada evento en benigno/sospechoso/malicioso con probabilidades, pistas MITRE ATT&CK y decisión operativa en <50 ms. Presentamos la motivación, dos casos de literatura verificados (Devign, CVSS-BERT), el diseño de la solución en cascada (tabular → texto → grafo) con su diagrama, y el dataset con visualizaciones exploratorias.**

I. Introducción y motivación

Un Centro de Operaciones de Seguridad (SOC) vigila la infraestructura con decenas de productos (syslog, auditoría Windows, flujos VPC, telemetría endpoint). WitFoo Precinct correlaciona todo con 261 reglas en *incidentes* (señales + subgrafo de proveniencia + reporte). El problema a abordar es el **triage de alertas**: dado un evento, predecir su tier `benign` / `suspicious` / `malicious` con probabilidades calibradas, hints ATT&CK y decisión `escalar` vs. `monitorear` en <50 ms para rankear la cola.

Es importante por volumen (2M de eventos en 6 días, imposible a mano), por desbalance extremo (0.38 % malicioso: el accuracy miente, un siempre-benigno acierta 94 %), por el cambio de contexto del analista (necesita mensaje + MITRE + cadena, no un número), porque las etiquetas serán correlación automática imperfecta (Unprocessed en 2,005,877/2,011,674), y porque hay deriva temporal (malicious termina 07-30, benigno sigue a 08-01: el split deberá ser temporal, no shuffle).

II. Revisión de literatura

Caso 1 — Devign (Zhou et al., NeurIPS 2019): GNN (GGNN + módulo Conv) sobre grafos de código (AST + dependencias), binaria a nivel función con datasets manuales (∼600 h-persona, Linux/QEMU/Wireshark/FFmpeg): **+10.51 % accuracy / +8.68 % F1** vs baselines, +27.99 % F1 vs analizadores estáticos, 74.11 % en 40 CVEs nuevos. Lección: precedente GNN del componente de grafo que planeamos; contraste: ellos etiquetas caras/limpias, nosotros Precinct baratas/ruidosas.

Caso 2 — CVSS-BERT (Shahid & Debar, IEEE ICMLA 2021): 8 clasificadores BERT (uno por métrica CVSS v3.1) desde la descripción del CVE (45,926 CVEs 2018–2020): accuracy 83.79–96.07 %, severidad MAE 0.73 (55.3 % match exacto), con saliency explicable. Límite: el CVSS varía según quién asigna. Lección: precedente texto→severidad del componente de texto que planeamos (TF-IDF primero, MiniLM después); su saliency inspira nuestra explicabilidad; su advertencia funda nuestras limitaciones previstas.

Cuadro I
Casos vs. proyecto planeado.

	Devign	CVSS-BERT	Nosotros (plan)
Input	grafo código	texto CVE	tabular + texto + subgrafo
Etiquetas	manuales 600 h	NVD experto	Precinct auto
Modelo	GGNN+Conv	8×BERT	LightGBM / TF-IDF+XGB / vecindario+X
Señal	+10.51 % acc	MAE 0.73	cascada M1→M2→M3 + explicar

Marco adicional a explorar: provenance-graph IDS (NODLINK, KnowHow, FLASH, threaTrace/GraphSAGE), CTI+LLM (OMNISEC, PROVSYN, TRAM con notebooks SciBERT), imbalance (Focal Loss, cost-sensitive: FN≫FP), TF-IDF que gana a BERT con poco dato, y CybORG (despliegue como agente SOC). Detalle y links en `docs/referencias_primer_entregable.md` y `docs/casos_primera_entrega.md`.

III. Solución y proceso (plan)

Planeamos: `signals` → prepro P1–P5 (one-hot sanitizado; temporales + split temporal anti-leakage; anti-desbalance sin SMOTE; TF-IDF 5k; subgrafo ±5 min) → M1 tabular que rankea / M2 de texto que confirma / M3 de grafo que confirma por cadena → fusión con regla de triage → consola (cola, incidente, cobertura MITRE).

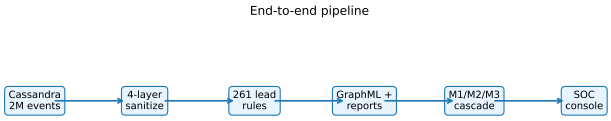


Figura 1. Diagrama esquemático end-to-end (requerido).

Evaluaremos con split temporal 80/20 (pasado entrena, futuro prueba), métrica F1-macro + PR-AUC por clase (nunca accuracy) y análisis de complejidad por modelo. El despliegue previsto es FastAPI en :8000 (`/score`, `/queue`, `/incident`) + consola React en el mismo origen (Docker multi-stage), con bitácora de feedback para re-entrenar.

IV. Dataset: explicación y visualización

`witfoo/precinct6-cybersecurity` v2.1.0 (Apache-2.0): captura live 2024-07-26 → 08-01, 4 orgs, 19 productos / 12 vendors, 38 columnas por señal, más

`incident_signals` histórico (238,511), 47,585 nodos / 1,740,438 aristas y 51,371 incidentes con reporte.

Cuadro II
TIERS LIVE (IR $\approx$ 245:1).

Tier	Filas	%
benign	1,897,153	94.31
suspicious	106,793	5.31
malicious (1,584 incidentes)	7,728	0.38

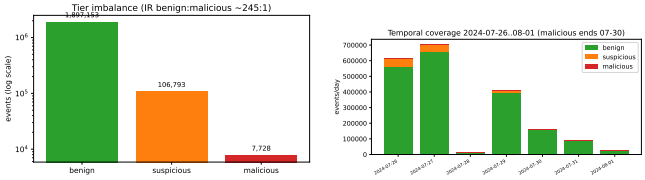


Figura 2. Exploración preliminar: desbalance (log) y cobertura temporal (malicious termina 07-30 $\rightarrow$ split temporal).

La exploración preliminar (`src/eda_report.py` $\rightarrow$ `reports/eda/`) muestra el diccionario de 38 columnas, `severity` 49 % vacía (= nivel propio, no mediana), priors ATT&CK que deberán excluirse de features (leakage), y tres hipótesis a verificar: tokens T1190 $\rightarrow$ componente de texto; vecinos ± 5 min $\rightarrow$ componente de grafo; tier medio recuperable tabular.

REFERENCIAS

[1] Y. Zhou *et al.*, "Devign: effective vulnerability identification by learning comprehensive program semantics via graph neural networks," in *NeurIPS*, 2019. arXiv:1909.03496.

[2] M. R. Shahid y H. Debar, "CVSS-BERT: explainable NLP to determine the severity of a computer security vulnerability from its description," in *IEEE ICMLA*, 2021. arXiv:2111.08510.

[3] WitFoo, Inc., "Precinct6 cybersecurity dataset v2.1.0," HuggingFace, 2026.

[4] MITRE, "ATT&CK design and philosophy," 2020. `attack.mitre.org`.

[5] D. Huang *et al.*, "NODLINK: online APT detection," in *NDSS*, 2024. arXiv:2311.02331.

[6] MITRE CTID, "Threat report ATT&CK mapper (TRAM)," `github.com/center-for-threat-informed-defense/tram`.

[7] T.-Y. Lin *et al.*, "Focal loss for dense object detection," in *ICCV*, 2017.

[8] W. Hamilton *et al.*, "Inductive representation learning on large graphs," in *NeurIPS*, 2017.

[9] M. Stanton *et al.*, "CybORG: a gym for cyber operations," arXiv:2002.10667, 2020.